Estruturas de dados para conjuntos disjuntos

Ideias e intuição por trás das análises

Jair

CMCC – UFABC

Para onde vamos

Queremos manter uma partição de $\{1, \ldots, n\}$ com três operações: `cria`, `busca` (quem é o representante do meu conjunto?) e `uniao` (junte dois conjuntos).

Compararemos quatro estruturas pelo **tempo total de** *m* **operações**, e não pelo pior caso de cada uma.

estrutura	**ideia em uma frase**
vetor	cada elemento guarda o nome do seu conjunto; unir é *renomear*
listas ligadas	renomear só o conjunto **menor**
floresta	unir é pendurar uma árvore em outra; buscar é *subir*
floresta + rank + compressão	subir também **achata** a árvore

Motivação: o algoritmo de Kruskal.

Kruskal: por que conjuntos disjuntos?

Enquanto o Kruskal roda, (V, S) é uma floresta; seus componentes formam uma **partição** de V.

A observação-chave

Uma aresta $\{x, y\}$ pode ser inserida sem criar circuito $\iff$ x e y estão em **componentes diferentes**.

Por quê? Se estão no mesmo componente, já há um caminho entre eles, e a aresta o fecha num circuito. Se estão em componentes diferentes, não há caminho entre eles, e qualquer circuito novo teria que passar pela aresta e voltar por um caminho que não existe.

Então o Kruskal só precisa de: *"x e y estão no mesmo conjunto?"* (duas buscas) e *"junte os dois conjuntos"* (uma união).

Kruskal com conjuntos disjuntos

Entrada: grafo conexo $G = (V, E)$ com pesos nas arestas
1: $S \leftarrow \emptyset$
2: $F \leftarrow$ lista das arestas em ordem crescente de peso
3: **para cada** $v \in V$ **faça**
4: `cria`(v)
5: **fim para**
6: **para cada** $\{u, v\} \in F$ **faça**
7: **se** `busca`$(u) \neq$ `busca`(v) **então**
8: insira $\{u, v\}$ em S
9: `uniao`(u, v)
10: **fim se**
11: **fim para**
12: **devolva** (V, S)

Contagem das operações

- $|V|$ operações `cria`;
- no máximo $2|E|$ operações `busca`;
- no máximo $|V| - 1$ operações `uniao`.

Total: menos que $2(|E| + |V|) = O(|E|)$, pois G é conexo e $|V| \leq |E| + 1$.

Consequência

Com $m = O(|E|)$ operações, o custo do Kruskal é

$$O(|E| \log |E|) \text{ (ordenação)} + \text{custo de } m \text{ operações.}$$

Como $|E| \leq |V|^2$, $O(|E| \log |E|) = O(|E| \log |V|)$. A estrutura escolhida decide se a ordenação domina.

Vetor: unir é renomear

Ideia: cada elemento guarda o *nome* do seu conjunto (o menor elemento dele). Dois elementos estão juntos se e só se têm o mesmo nome.

- `busca`(x): ler o nome de x — $O(1)$.
- `uniao`(x, y): **varrer o vetor inteiro** e trocar o nome dos dois conjuntos pelo menor deles — $\Theta(n)$.

Custo

Cada união varre n posições e há no máximo $n - 1$ uniões, logo m operações custam $O(m + n^2)$. No Kruskal: $O(|E| \log |E| + |V|^2)$, que em grafos esparsos é dominado por $|V|^2$.

O desperdício: para unir um conjunto minúsculo com um enorme, renomeamos *todos* os elementos, inclusive os que já estavam certos.

Listas ligadas: renomear só o menor

Ideia: cada conjunto é uma lista; cada nó aponta para o representante (`rep`).

- `busca`(x): seguir `rep` — $O(1)$.
- `uniao`(x, y): **percorrer a lista menor**, apontando seus nós para o representante da maior, e concatenar — custo proporcional ao tamanho da menor.

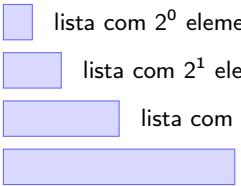
Heurística (‡)

Numa união, atualize sempre os ponteiros da **menor** lista.

Sem (‡) podemos gastar $\Theta(n^2)$ (sempre renomear a lista maior); com (‡) a análise a seguir mostra $O(m + n \log n)$.

Listas ligadas: quem paga a atualização?

Olhe a história de **um elemento** x. Ele só tem o seu ponteiro atualizado quando está na lista **menor**, e então passa a estar numa lista **pelo menos duas vezes maior**:



Como nenhuma lista passa de n elementos, x é atualizado no máximo $\log_2 n$ vezes. **Cada um dos n elementos paga no máximo** $\log_2 n$; o resto das operações custa $O(1)$.

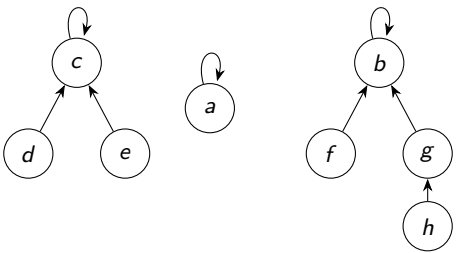
Resultado

m operações custam $O(m + n \log n)$; o Kruskal fica $O(|E| \log |V|)$. (A prova formal está no apêndice.)

Floresta: cada conjunto é uma árvore

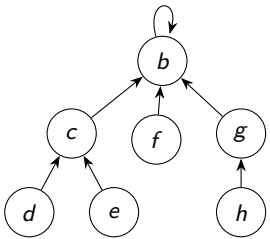
Ideia: em vez de avisar *todos* os elementos, avise só a raiz. Cada elemento aponta para o pai; a raiz é o representante.

- `uniao`: pendurar uma raiz na outra — $O(1)$ depois de achar as raízes.
- `busca`(x): **subir** até a raiz — custa a profundidade de x.



Os conjuntos $\{c, d, e\}$, $\{a\}$ e $\{b, f, g, h\}$.

Floresta: o risco e o remédio



`uniao`(e, f): a raiz c (tamanho 3) é pendurada na raiz b (tamanho 4).

Risco: pendurar sempre a árvore *grande* numa unitária cria uma cadeia de n nós; uma busca custa $\Theta(n)$.

Remédio: pendurar a menor na maior

Por **tamanho** (número de nós) ou por **altura**: quem é pendurado fica um nível mais fundo, então queremos que seja a árvore que "tem menos a perder".

Floresta: algoritmos com união por tamanho

Cada nó tem $\mathrm{pai}[x]$ e, nas raízes, $t[x]$ = número de elementos da árvore enraizada em x.

cria(x)

1: $\mathrm{pai}[x] \leftarrow x$
2: $t[x] \leftarrow 1$

busca(x)

1: **enquanto** $x \neq \mathrm{pai}[x]$ **faça**
2: $\quad x \leftarrow \mathrm{pai}[x]$
3: **fim enquanto**
4: **devolva** x

link(x, y)

1: $t[x] \leftarrow t[x] + t[y]$
2: $\mathrm{pai}[y] \leftarrow x$

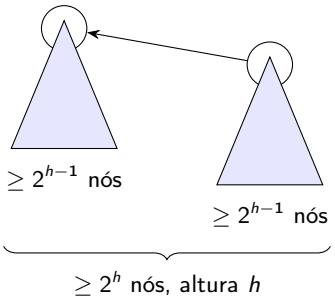
`link` pressupõe que x e y são raízes distintas com $t[x] \geq t[y]$ (garantido por `uniao`); `uniao` pressupõe que x e y estão em árvores distintas.

uniao(x, y)

1: $r_x \leftarrow$ `busca`(x); $\quad r_y \leftarrow$ `busca`(y)
2: **se** $t[r_x] \geq t[r_y]$ **então**
3: $\quad$ `link`(r_x, r_y)
4: **senão**
5: $\quad$ `link`(r_y, r_x)
6: **fim se**

Floresta: por que a altura fica pequena?

Ideia: a altura de uma árvore só aumenta quando pendurada uma árvore de *mesma altura*. Portanto, uma árvore de altura h foi montada a partir de **duas de altura** $h-1$:



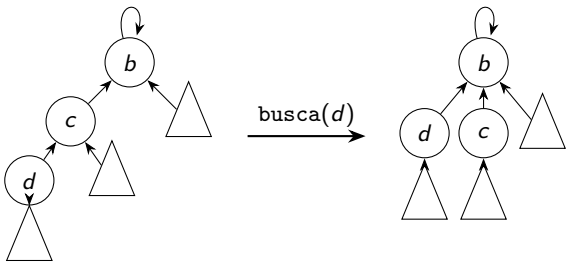
Logo uma árvore de altura h tem $\geq 2^h$ nós; com no máximo n nós, a altura é $\leq \log_2 n$. Cada busca custa $O(\log n)$ e

Resultado

m operações custam $O(m(1+\log n))$; o Kruskal fica $O(|E|\log|V|)$.

Compressão de caminhos: subir também arruma

Ideia: uma busca já percorreu o caminho de x até a raiz; aproveitamos a viagem e fazemos **todos os nós do caminho apontarem direto para a raiz**. A próxima busca por qualquer um deles custa $O(1)$.



Uma busca *cara* se paga: ela deixa a árvore mais achatada para as seguintes. O desafio é **contar isso direito**.

Rank: a altura que a árvore teria sem compressão

A compressão encurta caminhos, e manter a altura exata a cada busca seria caro. Em vez disso guardamos em cada raiz um **rank**:

- começa em 0 e só muda numa união: se as duas raízes têm o mesmo rank, a que recebe sobe 1;
- é uma **cota superior** da altura (igual a ela quando não há compressão);
- um nó de rank r tem pelo menos 2^r descendentes, como no argumento da altura: então $\mathrm{rank} \leq \log_2 n$.

Duas propriedades que usaremos

(a) **Quem sobe, sobe de rank:** todo ancestral próprio de x tem rank **estritamente maior** que o de x.

(b) Depois que deixa de ser raiz, o rank de um nó **nunca mais muda**; o rank do *pai* de x só pode aumentar (ou ficar igual) ao longo do tempo.

Rank e os algoritmos

cria(x)

1: $\text{pai}[x] \leftarrow x$
2: $\text{rank}[x] \leftarrow 0$

busca(x) **com compressão**

1: **se** $x \neq \text{pai}[x]$ **então**
2: $\quad \text{pai}[x] \leftarrow \texttt{busca}(\text{pai}[x])$
3: **fim se**
4: **devolva** $\text{pai}[x]$

link(x, y)

1: **se** $\text{rank}[x] = \text{rank}[y]$ **então**
2: $\quad \text{rank}[x] \leftarrow \text{rank}[x] + 1$
3: **fim se**
4: $\text{pai}[y] \leftarrow x$

uniao(x, y)

1: $r_x \leftarrow \texttt{busca}(x)$; $r_y \leftarrow \texttt{busca}(y)$
2: **se** $\text{rank}[r_x] \geq \text{rank}[r_y]$ **então**
3: $\quad \texttt{link}(r_x, r_y)$
4: **senão**
5: $\quad \texttt{link}(r_y, r_x)$
6: **fim se**

`link` pressupõe que x e y são raízes distintas com $\text{rank}[x] \geq \text{rank}[y]$ (garantido por `uniao`), e `uniao` que x e y estão em árvores distintas.

O que se consegue

Teorema (Tarjan, 1975)

Com união por rank e compressão de caminhos, m operações ($m \geq n$) custam $O(m\,\alpha(m, n))$, onde α é a inversa da função de Ackermann.

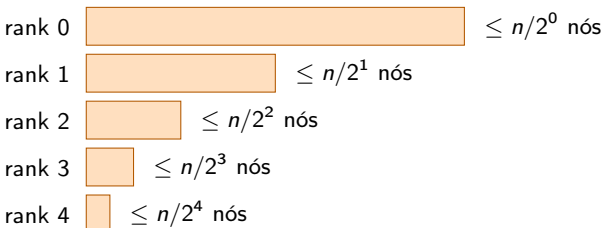
Quão lenta é α? A função de Ackermann cresce absurdamente rápido; sua inversa, absurdamente devagar:

n	$\alpha(n, n)$
$1 - 3$	1
$4 - 15$	2
$16 - 65535$	3
$65536 - 2^{2^{2^{\cdots}}}$ (torre de 17 dois)	4

Na prática, $\alpha \leq 4$: é como se cada operação custasse tempo constante. Provaremos uma cota um pouco mais fraca, $O((m + n) \lg^* n)$, com a mesma ideia. ($\lg^* n$: quantas vezes é preciso tirar lg até chegar a 1; para $n \leq 2^{65536}$, vale $\lg^* n \leq 5$.)

Análise (1/3): poucos nós têm rank alto

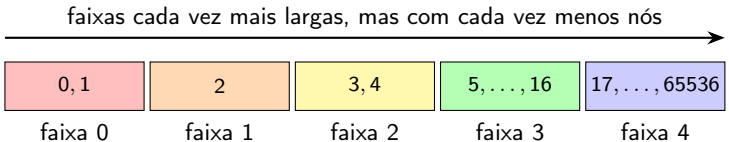
Um nó de rank r "leva consigo" pelo menos 2^r elementos, e esses elementos **não são reaproveitados** por outro nó do mesmo rank. Logo, há no máximo $n/2^r$ nós de rank r:



É uma **pirâmide**: metade dos nós tem rank baixo, e os ranks altos são raros. Em particular, os nós de rank maior que j são no máximo $n/2^j$.

Análise (2/3): agrupar os ranks em faixas

Dividimos os ranks possíveis $(0, \ldots, \log_2 n)$ em **faixas** que ficam cada vez mais largas: a faixa g cobre os ranks $(K, 2^K]$, com K o fim da faixa anterior.



A largura **explode** (de K para 2^K), então bastam $\lg^* n$ faixas para cobrir todos os ranks. **Compromisso:** faixa mais larga $\Rightarrow$ cada nó pode pagar mais vezes, mas a pirâmide garante que há menos nós nela.

Análise (3/3): quem paga cada busca?

Numa busca, um nó y do caminho paga $O(1)$. Separamos os pagantes:

Tipo 1: a raiz, o filho da raiz, ou quem **troca de faixa** para o pai

No caminho o rank só cresce, então o caminho cruza no máximo $\lg^* n$ faixas: $\leq \lg^* n + 1$ **nós por busca**, em $O(m \lg^* n)$ ao todo.

Tipo 2: y e o pai estão na **mesma** faixa

Cada vez que y paga, ele passa a apontar para a raiz, que tem rank **maior** que o do pai anterior. Como a faixa tem no máximo 2^K ranks, y paga no máximo 2^K vezes e depois seu pai sai da faixa. A faixa tem no máximo $n/2^K$ nós:

$$\underbrace{\frac{n}{2^K}}_{\text{nós}} \cdot \underbrace{2^K}_{\text{pagamentos}} = n \qquad \Rightarrow \qquad \text{cada faixa custa } n.$$

Com $\lg^* n$ faixas: $O(n \lg^* n)$.

Resumo

Estrutura	**Quem paga e quanto**	m **operações**
Vetor	cada união paga n	$O(m + n^2)$
Listas, menor lista	cada elemento paga $\leq \log n$	$O(m + n \log n)$
Floresta (tam./altura)	cada busca paga $\leq \log n$	$O(m(1 + \log n))$
Floresta + compressão	cada faixa de rank paga n	$O((m + n) \lg^* n)$

No Kruskal, com $m = O(|E|)$ e $n = |V|$, as três últimas deixam a ordenação das arestas, $O(|E| \log |V|)$, como o termo dominante.

Apêndice

Demonstrações formais

Listas ligadas: a heurística da menor lista

Atualizar sempre a menor lista (‡)

Numa união, os ponteiros `rep` são atualizados na menor das duas listas (em qualquer uma, se têm o mesmo tamanho), que é acrescentada ao final da maior; isto é, a menor lista faz o papel de L_y na descrição de `uniao`.

Intuição: cada vez que `rep`$[x]$ é atualizado, a lista de x pelo menos dobra de tamanho.

Proposição

Supondo (‡), em qualquer momento, se `rep`$[x]$ foi atualizado k vezes, então a lista que contém x tem pelo menos 2^k elementos.

Prova.

Os conjuntos só crescem, logo o tamanho da lista de x não diminui. Indução em k: para $k = 0$, a lista tem pelo menos $1 = 2^0$ elemento. Seja $k \geq 1$ e L a lista de x imediatamente antes da k-ésima atualização; por indução e monotonicidade, $|L| \geq 2^{k-1}$. Como `rep`$[x]$ é atualizado, por (‡) a outra lista L' tem $|L'| \geq |L|$, e a lista resultante tem $|L| + |L'| \geq 2|L| \geq 2^k$ elementos. □

União por tamanho: tamanho × altura

Proposição

Usando união por tamanho, e supondo que as n primeiras operações são `cria`, em qualquer momento cada árvore tem pelo menos 2^h nós, onde h é a sua altura.

Prova.

Indução no número k de uniões já realizadas. Para $k = 0$, as únicas operações até então que alteram a estrutura são `cria`, e toda árvore tem um nó e altura 0, com $1 \geq 2^0$. Seja $k \geq 1$: como as n primeiras operações são `cria`, nada além das uniões altera a estrutura. A k-ésima união liga duas árvores distintas A e B, com tamanhos t_A, t_B e alturas h_A, h_B; por indução $t_A \geq 2^{h_A}$ e $t_B \geq 2^{h_B}$. Sem perda de generalidade $t_A \geq t_B$ e a raiz de B é pendurada na de A. A árvore resultante tem $t' = t_A + t_B$ nós e altura $h' = \max\{h_A, h_B + 1\}$.

- Se $h' = h_A$: $t' \geq t_A \geq 2^{h_A} = 2^{h'}$.
- Se $h' = h_B + 1$: $t' = t_A + t_B \geq 2t_B \geq 2 \cdot 2^{h_B} = 2^{h'}$.

□

Um resultado mais fraco: grupos de *rank*

Considere m operações, as n primeiras `cria`, com união por *rank* e compressão. Sejam $b_0 = 1$ e $b_g = 2^{b_g - 1}$, isto é, $1, 2, 4, 16, 65536, \ldots$

Particionamos $\mathcal{U}$ em **grupos** segundo o *rank final*:

- Grupo 0: elementos de rank 0 e 1;
- Grupo $g \geq 1$: elementos de rank em $(b_{g-1}, b_g] = (K, 2^K]$, com $K = b_{g-1}$.

Propriedades do *rank*

(i) o rank de um nó só muda em `link`, quando ele é a raiz que recebe a outra; depois que deixa de ser raiz, não muda mais;

(ii) $\mathrm{rank}[x] < \mathrm{rank}[\mathrm{pai}[x]]$ para todo x que não é raiz; em particular, $\mathrm{rank}[\mathrm{pai}[x]]$ não decresce ao longo do tempo, e aumenta estritamente quando a compressão passa a apontar x para um novo pai;

(iii) se $x \neq y$ têm o mesmo rank final, então $D(x) \cap D(y) = \emptyset$, e $|D(x)| \geq 2^{\mathrm{rank}[x]}$, onde $D(x)$ é a árvore de x no instante em que x deixa de ser raiz (ou no fim);

(iv) para todo inteiro $j \geq 0$, há no máximo $n/2^j$ nós de rank maior que j; em particular (com $j = K = b_{g-1}$), o Grupo g tem no máximo $n/2^K$ nós;

(v) há no máximo $\lg^* n$ grupos.

Justificativa de (ii) e (iii)

(ii) Vale na criação; em `link`, se os ranks são iguais o da raiz que recebe aumenta; e a compressão aponta x para um ancestral, que tem rank maior por transitividade de (ii) ao longo do caminho até a raiz. Logo $\mathrm{rank}[\mathrm{pai}[x]]$ não decresce.

(iii) A árvore de x no instante em que deixa de ser raiz tem pelo menos $2^{\mathrm{rank}[x]}$ nós (proposição com rank), e o seu rank já é o final por (i).
Suponha $z \in D(x) \cap D(y)$, com $x \neq y$ do mesmo rank r e, sem perda de generalidade, x deixando de ser raiz antes de y (ou y sendo raiz no fim).

- Nesse instante x é ligado a uma raiz R de rank $\geq r$; após o `link` a raiz da árvore de z passa a ter rank $> r$ (se $\mathrm{rank}[R] = r$, ele aumenta), e esse rank não decresce.
- Quando y deixa de ser raiz, y é a raiz da árvore de z com rank r: contradição.

Justificativa de (iv) e (v)

(iv) Para cada r, os $D(x)$ de rank r são disjuntos e têm $\geq 2^r$ elementos, logo há $\leq n/2^r$ nós de rank r. Somando sobre $r > j$:

$$\sum_{r>j} \frac{n}{2^r} = \frac{n}{2^j}.$$

(v) O rank é no máximo $\lfloor \lg n \rfloor$ (proposição). Se g_0 é o menor g com $b_g \geq \lfloor \lg n \rfloor$, há $g_0 + 1$ grupos. Para $g_0 \geq 1$ vale $b_{g_0-1} < \lg n$, isto é, $n > 2^{b_{g_0-1}} = b_{g_0}$. Aplicando lg repetidamente,

$$\lg^{(g_0)} n > \lg^{(g_0)} b_{g_0} = b_0 = 1,$$

logo $\lg^* n \geq g_0 + 1$.

Custo das buscas: nós de dois tipos

O custo total é $O(m)$ (`cria` e `link` custam $O(1)$) mais o custo das $f \leq 2m$ buscas. O custo de uma busca é o número de nós do caminho $x = y_0, \ldots, y_k$ até a raiz, cujos apontadores são redirecionados. Classificamos os nós y do caminho:

1. **Tipo 1:** y é a raiz ou filho da raiz, ou $\mathrm{pai}[y]$ está em grupo diferente do de y;
2. **Tipo 2:** os demais, isto é, y não é a raiz nem filho da raiz e $\mathrm{pai}[y]$ está no mesmo grupo que y.

Tipo 1

Ao longo do caminho o rank cresce estritamente, logo o grupo não decresce e muda no máximo $\lg^* n - 1$ vezes; com a raiz e o filho da raiz, cada busca tem no máximo $\lg^* n + 1$ nós do Tipo 1. Em f buscas são

$$O(m \lg^* n).$$

Nós do Tipo 2

Seja y no Grupo $g \geq 1$, com ranks em $(K, 2^K]$. Cada vez que y é do Tipo 2 em uma busca, $\mathrm{pai}[y]$ passa a ser a raiz, cujo rank é maior que o do pai anterior (pois este não era a raiz). Como o rank do pai não decresce e o de y é fixo por (i):

- o rank de $\mathrm{pai}[y]$ assume valores distintos em $(\mathrm{rank}[y], 2^K]$ a cada ocorrência;
- depois que sai do grupo, y nunca mais é do Tipo 2.

Logo y é do Tipo 2 no máximo 2^K vezes. Por (iv) há no máximo $n/2^K$ tais nós, e o Grupo g contribui com no máximo

$$\frac{n}{2^K} \cdot 2^K = n.$$

No Grupo 0 cada nó é do Tipo 2 no máximo uma vez (o pai precisa ter rank 1, e depois o pai passa para fora do grupo): contribui com no máximo n. Somando sobre os $\leq \lg^* n$ grupos,

$$O(n \lg^* n).$$